



An advanced deep reinforcement learning algorithm for three-layer D2D-edge-cloud computing architecture for efficient task offloading in the Internet of Things

Komeil Moghaddasi^a, Shakiba Rajabi^a, Farhad Soleimanian Gharehchopogh^{a,*}, Ali Ghaffari^{b,c,d}

^a Department of Computer Engineering, Urmia Branch, Islamic Azad University, Urmia, Iran

^b Department of Computer Engineering, Tabriz Branch, Islamic Azad University, Tabriz, Iran

^c Department of Computer Engineering, Faculty of Engineering and Natural Science, Istinye University, Istanbul, Turkey

^d Department of Computer Science, Khazar University, Baku, Azerbaijan

ARTICLE INFO

Keywords:

Internet of Things
Task offloading
Device-to-Device communications
Edge computing
Cloud computing

ABSTRACT

The Internet of Things (IoT's) has transformed the digital landscape by interconnecting billions of devices worldwide, paving the way for smart cities, homes, and industries. With the exponential growth of IoT devices and the vast amount of data they generate, concerns have arisen regarding efficient task-offloading strategies. Traditional cloud and edge computing methods, paired with basic Machine Learning (ML) algorithms, face several challenges in this regard. In this paper, we propose a novel approach to task offloading in a Device-to-Device (D2D)-Edge-Cloud computing using the Rainbow Deep Q-Network (DQN), an advanced Deep Reinforcement Learning (DRL) algorithm. This algorithm utilizes advanced neural networks to optimize task offloading in the three-tier framework. It balances the trade-offs among D2D, Device-to-Edge (D2E), and Device/Edge-to-Cloud (D2C/E2C) communications, benefiting both end users and servers. These networks leverage Deep Learning (DL) to discern patterns, evaluate potential offloading decisions, and adapt in real time to dynamic environments. We compared our proposed algorithm against other state-of-the-art methods. Through rigorous simulations, we achieved remarkable improvements across key metrics: an increase in energy efficiency by 29.8%, a 27.5% reduction in latency, and a 43.1% surge in utility.

1. Introduction

Artificial intelligence lets computers mimic human smarts, solving problems and making smart decisions [1]. It is key in IoT, revolutionizing how devices talk and work together in our digital world. Spanning across diverse sectors such as healthcare [2], transformative landscape [3], robotics [4], industrial waste management [5], agriculture, and urban infrastructures, IoT systems promise transformative solutions, bolstered by automation and real-time data analytics. This extensive network of devices and applications is transforming industries, enhancing urban intelligence, and presenting novel solutions to long-standing challenges [6]. However, the rise of IoT brings forth pressing issues. Primarily, the exponential data generation by countless devices poses significant challenges for traditional computing models [7]. Every day, a large amount of data is produced, and efficiently processing this voluminous information, especially in scenarios demanding real-time

responses, is paramount [8]. The sheer scale of data transmission and processing has begun to strain centralized computing infrastructures, prompting the need for novel computational strategies [9].

In this context, task offloading has emerged as a pivotal solution. It allows IoT ecosystems to achieve a balanced operational milieu, characterized by enhanced energy efficiency, minimized latency, and optimized performance [10]. Furthermore, the advent of edge computing has amplified the potential of offloading [11]. By positioning data processing closer to the source, either within the IoT devices or on proximate intermediary nodes, edge computing ensures a swifter response to dynamic data needs, thus reducing the latency traditionally associated with distant centralized servers [12,13]. Edge nodes, despite their advantages, are not a universal solution [14]. Intricate computational tasks or those demanding vast storage often surpass the capabilities of edge nodes [15,16]. Here, cloud computing with its expansive computational power and storage capacities, becomes indispensable [17]. The cloud,

* Corresponding author.

E-mail address: bonab.farhad@gmail.com (F.S. Gharehchopogh).

<https://doi.org/10.1016/j.suscom.2024.100992>

Received 28 January 2024; Received in revised form 26 March 2024; Accepted 29 April 2024

Available online 1 May 2024

2210-5379/© 2024 Published by Elsevier Inc.

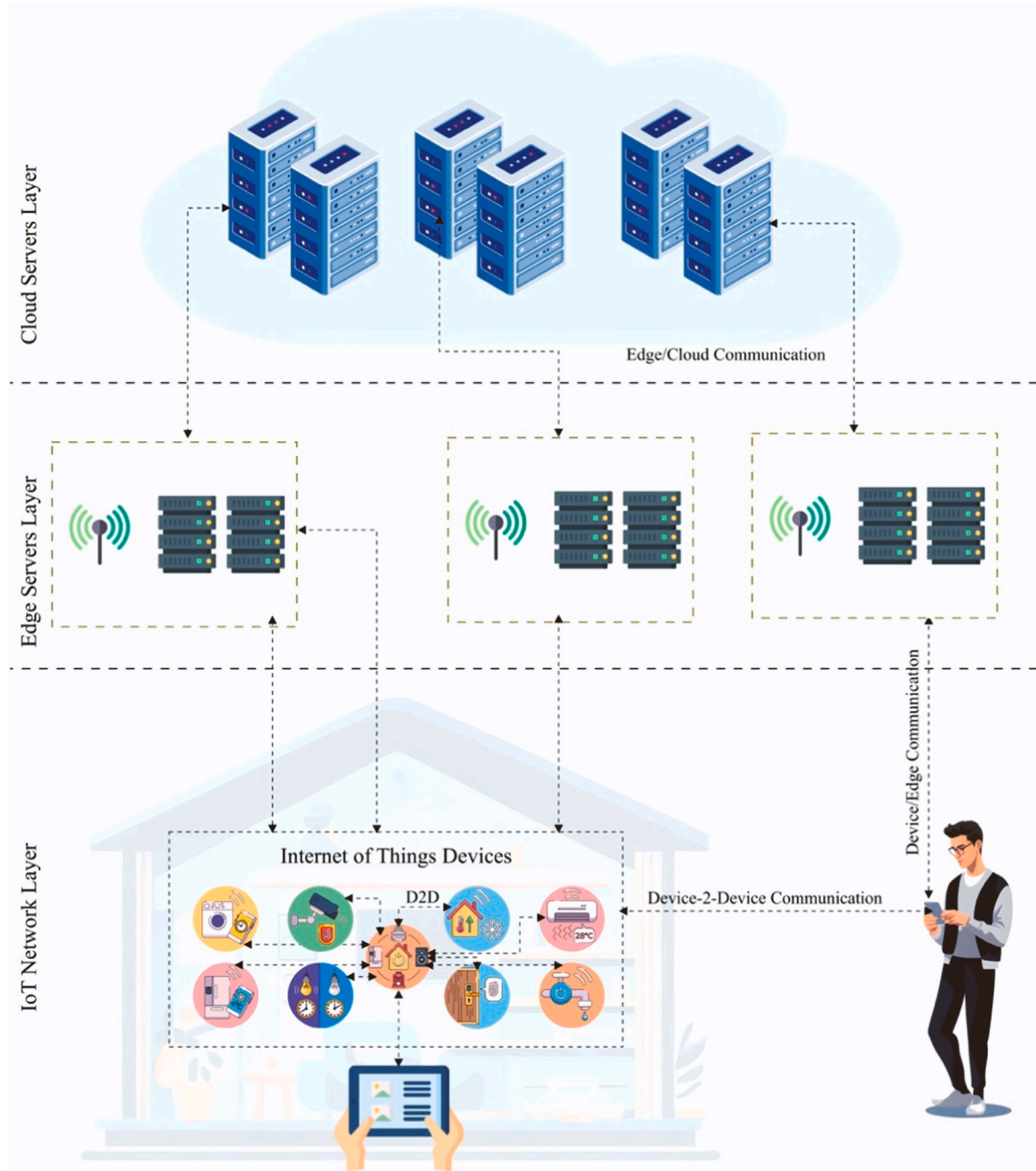


Fig. 1. IoT architecture showcasing D2D, edge, and cloud layers with interlayer communication for task offloading.

with its distributed architecture and scalable resources, can accommodate the hefty processing needs that the IoT often demands [18]. Our work is motivated by the need to optimize task offloading across the D2D, Edge, and Cloud spectrum to enhance operational efficiency in IoT systems.

Amid the landscape of IoT research, efficient task offloading strategies have been central, with a spectrum of approaches ranging from heuristic to machine learning-based models. Despite the contribution of these methods, a critical gap exists the ability to dynamically navigate the fluctuating network conditions and concurrently optimize across the D2D, Edge, and Cloud computing tiers. Addressing this shortfall, we propose the integration of the Rainbow DQN, a sophisticated DRL algorithm. The proposed method, distinguished by its combination of six advanced enhancements, offers a significant breakthrough in terms of stability and efficiency for complex decision-making scenarios. Our adoption of Rainbow DQN is characterized by its innovative and

requisite nature for IoT task offloading, ushering in an adaptive and real-time decision-making paradigm. This shift is critical in transcending the static and non-adaptive limitations of extant algorithms, such as the comparison algorithms in this paper, metaheuristic algorithms, conventional DQN, and Monte Carlo methods, which do not suffice in the dynamically evolving IoT environment marked by uncertainty and swift network status changes. The key contributions of our research include:

- We present a method for task offloading across the D2D, Edge, and Cloud spectrum, crucial for elevating operational efficiency in IoT systems.
- We incorporate the Rainbow DQN, an advanced DRL algorithm. Enhanced by its deep neural network structures, this model offers a resilient mechanism for pinpointing optimal offloading decisions amidst fluctuating conditions.

Table 1

Properties of the related schemes and proposed scheme.

Reference	Utilized Algorithm	Evaluation metrics			Approach				Operation methods			
		Energy	Latency	Utility	End-Device	End-Server	With Cloud	Without Cloud	Recourse allocation	Load balancing	Scheduling	Offloading
[19]	Heuristic-based	✓			✓			✓	✓			
[20]	Heuristic-based				✓			✓				✓
[21]	RL-based				✓			✓	✓			
[22]	FL-based				✓	✓		✓	✓			
[23]	Game theory-based	✓	✓		✓	✓		✓				✓
[24]	Heuristic-based				✓			✓				
[25]	ML-based					✓	✓					
[26]	Neural-network-based					✓	✓					
[27]	DRL-based					✓	✓		✓			
[28]	DRL-based		✓			✓	✓		✓			
[29]	DRL-based			✓		✓	✓					
This Paper	DRL-based	✓	✓	✓	✓	✓	✓					✓

- We contrast our methodology with leading state-of-the-art algorithms, demonstrating the preeminence of our strategy in the arena of task offloading.
- Through meticulous simulations, we establish compelling evidence of noteworthy improvements in key performance indicators.

Fig. 1 illustrates the hierarchical structure of a D2D-Edge-Cloud computing architecture within the context of IoT networks. At the lowest tier, IoT devices engage in direct communication, above which the edge servers layer facilitates closer computational resources. The topmost cloud servers layer provides extensive data processing capabilities, with bidirectional communication links connecting these layers to enable efficient task offloading and data flow.

Following this introduction, the next section provides an overview of current research in the field, outlining existing methods and identifying their shortcomings. We then proceed to describe our system's structure and the proposed offloading method. Following the description of our experimental design, we analyze the results to provide insights into the performance of our method. Also, we discuss these results in detail, comparing our model to existing offloading methods and pointing out its benefits. Finally, the paper concludes with a summary of our findings, the implications for the D2D-Edge-Cloud computing environment, limitations, and potential directions for future research in this field.

2. Related works

In this section, we explore studies that focus on the integration of IoT and related applications within the D2D, Edge, and cloud computing frameworks. To start with, Su et al. [19] examined D2D communications in UAV-assisted IIoTs, emphasizing the integration of simultaneous wireless information and power transfer (SWIPT). Their goal was to enhance system energy efficiency without violating transmission rate and power budget constraints. By employing the Dinkelbach method and MOEA/D optimization, they managed to transform complex optimization challenges into more manageable forms. Numerical results showcased the efficacy of their approach, revealing substantial energy efficiency improvements over benchmark methods., their scheme primarily focused on UAV-assisted networks, which may limit its applicability to other IIoT settings. In another study, Ibrar et al. [20] introduced the ToSIIoT scheme, designed to augment resource utilization and the quality of service in D2D-based social IIoT settings. Their approach considered both the device utilization ratio and the strength of social relationships to optimize task completion. The solution they offered demonstrated superior metrics in resource utilization, quality of service, and task completion compared to existing methods. The scheme's reliance on social relationships might restrict its usability in non-social

IIoTs. Emphasizing the integration of resource allocation and drone relay selection, Liu et al. [21] sought to enhance the sum rate of D2D systems, maintaining QoS for both Cellular Users (CUs) and D2D Users (DUs). They approached resource allocation with a two-phase coalitional game strategy, blending both Visible Light Communication (VLC) and Radio Frequency (RF) technologies. The potential scalability issues in expansive D2D networks could arise from the complexity of their two-phase strategy. In their publication, Guo et al. [22] presented a D2D communication-supported Digital Twin Edge Network scheme. This multifaceted approach integrated edge computing, digital twin technology, and Federated Reinforcement Learning. Their proposed scheme marked a significant stride forward in optimizing network performance for IoT devices. However, the intricate integration of multiple technologies might complicate its deployment in resource-limited environments.

Yang et al. [23] leveraged game theory to present a solution for computation offloading challenges in multi-user networks. Their "Game-Theoretic Offloading Method" ensured minimized individual task execution costs and overall system overhead. The approach's reliance on reaching a Nash Equilibrium might not be feasible in highly dynamic network scenarios. On 5 G NB-IoT D2D relaying, Gbadamosi et al. [24] proposed an interference-aware scheme. They employed the Max-SINR approach to optimize the selection of D2D relays, aiming to bolster the efficiency of the selection process. Potential limitations arise as their model may not fully consider real-world network dynamics and unexpected interference sources. Jia et al. [25] and Hafeez et al. [26] both centered their studies on fortifying IoT security. Jia et al.'s FlowGuard was tailored to counter IoT DDoS attacks using machine learning models. In contrast, Hafeez et al.'s IOT-KEEPER aimed to bolster IoT communication security through novel anomaly detection. However, as security threats continually evolve, both schemes may require regular updates to stay effective against newer attack vectors.

With a focus on dynamic task allocation between edge and cloud platforms, Chen et al. [27] presented the DTASM scheme for IoT systems. Leveraging DRL, their approach adapted to evolving user requirements and mobility trajectories. The inherent computational demands of their deep learning models could be challenging in real-time contexts. Cui et al. [28] proposed a cloud-edge cooperative content-delivery strategy tailored for IoV settings. Their scheme optimized content caching and request routing using DRL to ensure reduced network latency. The strategy might struggle in scenarios with rapidly changing content demands and network topologies.

Finally, addressing the optimization of gateway placement and multichip routing in IoT systems, Pham et al. [29] advanced approximation algorithms that not only offered strategic insights for IoT providers but also markedly reduced computational time. Into several

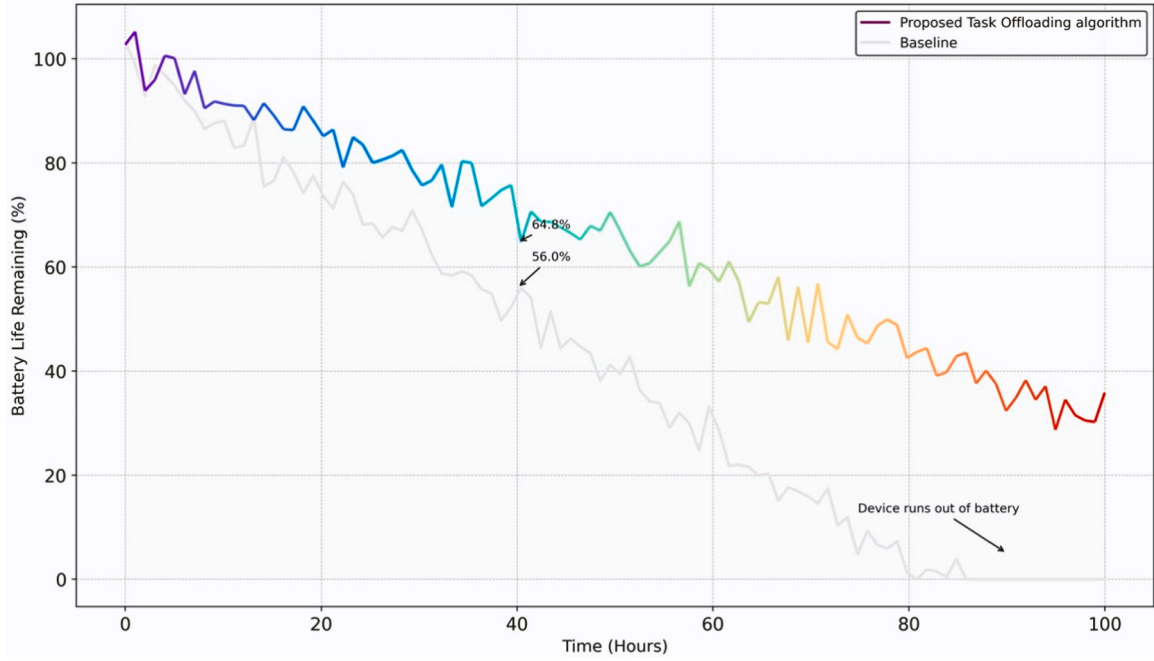


Fig. 2. Battery life comparison over time between the baseline and the proposed task offloading algorithm showing prolonged device endurance.

optimization, models could complicate their assimilation into cohesive IoT solutions.

In the landscape of task offloading across the D2D-Edge-Cloud continuum, the discussed related works predominantly focus on specific aspects, such as energy efficiency, security, routing optimization, or individualized offloading strategies within isolated contexts. These schemes often employ heuristic algorithms, game theory, or basic RL models to tackle their chosen challenges. In contrast, our study takes a holistic approach, introducing a pioneering task offloading strategy in a comprehensive three-tier D2D-Edge-Cloud environment. We harness the power of Rainbow DQN, an advanced DRL algorithm, to optimize task offloading across the entire spectrum. Our approach not only capitalizes on deep learning's capability to discern intricate patterns but also seamlessly adapts to dynamic environments. By navigating the trade-offs among D2D, D2E, and D2C/E2C communications, our strategy ensures mutual benefits for both end users and servers. When benchmarked against the state-of-the-art, our model outperforms them, registering significant improvements in energy efficiency, latency, and utility. Table 1 highlights the properties and differences of the related schemes in comparison to our study.

3. System model and proposed algorithm

In this section, we delve into a detailed breakdown of our system model. Our goal is to methodically illuminate the foundational principles, provide clear mathematical constructs, and elucidate the nuanced algorithmic intricacies that underpin our D2D-Edge-Cloud offloading approach. This detailed exposition ensures ease of comprehension and effective replication of our methodology.

3.1. Devices (D)

To initiate our exploration into the mechanics of the proposed system, let's begin by considering the individual devices. $P_D^i(t)$ represents the processing capability of device i at time t (Eq. (1)):

$$P_{\min}^D \leq P_D^i \leq P_{\max}^D \quad (1)$$

Energy consumption for local computation is given by Eq. (2):

$$E_{\text{comp}}^i(t) = k_1 \times (P_D^i(t)) \times T_{\text{exec}}^i(t) \quad (2)$$

Where k_1 is a device-specific constant, and $T_{\text{exec}}^i(t)$ is the execution time for a task on the device i at time t .

The rate of battery drains for device i at time t during task execution is defined as Eq. (3):

$$R_{\text{drain}}^i(t) = k_2 \times E_{\text{comp}}^i(t) + k_3 \times P_D^i(t) \quad (3)$$

Where k_2 and k_3 are constants.

Updated battery life after task execution (Eq. (4)):

$$B_D^i(t+1) = B_D^i(t) - R_{\text{drain}}^i(t) \times T_{\text{exec}}^i(t) \quad (4)$$

As we can see in Fig. 2, the graph illustrates how the proposed task offloading algorithm can extend the battery life of IoT devices. The comparison between the baseline and the algorithm shows a clear improvement in battery sustainability, delaying the point at which the device runs out of battery.

The position of device i at time t is according to Eq. (5).

$$Pos^i(t) = (x^i(t), y^i(t)) \quad (5)$$

The velocity of device i (only for mobile devices) is modeled as Eq. (6).

$$V^i(t) = \frac{\Delta Pos^i}{\Delta t} \quad (6)$$

Storage used by device i at time t (Eq. (7)).

$$S_{\text{used}}^i(t) = S_{\text{total}}^i - S_{\text{free}}^i(t) \quad (7)$$

The storage requirement for a task is given by Eq. (8).

$$S_{\text{req}}^i(t) = \alpha \times L_{\text{data}}^i(t) \quad (8)$$

According to Eq. (9), let $BW^i(t)$ be the available bandwidth for device i at time t .

$$BW_{\min} \leq BW^i(t) \leq BW_{\max} \quad (9)$$

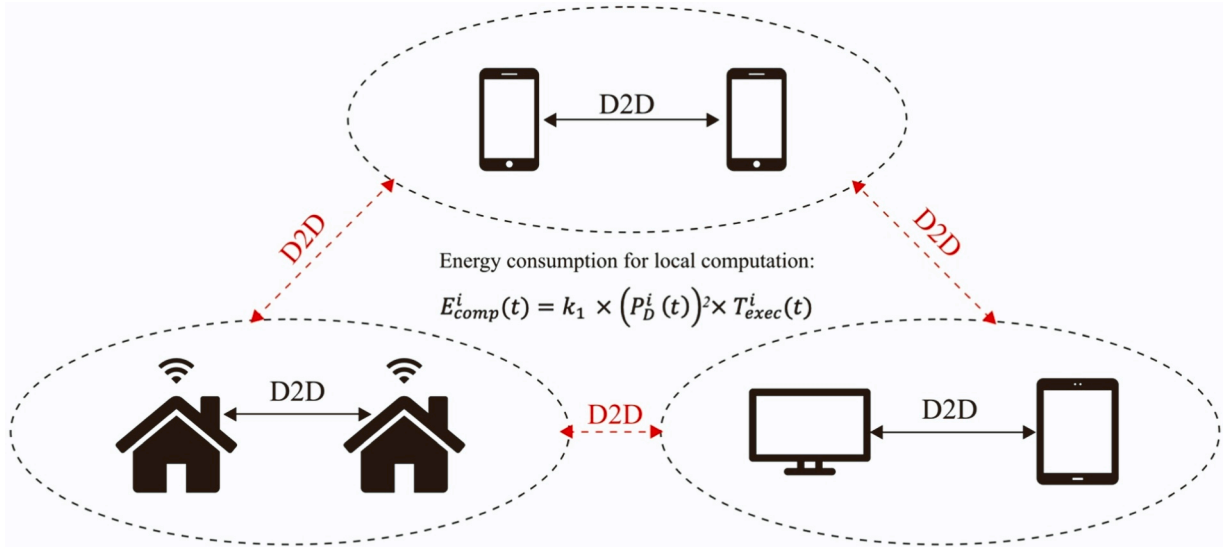


Fig. 3. Single-tier D2D communication links between various devices with the formula for calculating energy consumption during local computation.

3.2. Edge Nodes (E)

Intermediate computational entities are strategically located closer to the devices than centralized cloud servers.

The processing capability of edge node j at time t is influenced by a sigmoid function to model gradual saturation as defined in Eq. (10).

$$P_E^j(t) = P_{\max}^E \times \frac{1}{1 + e^{-k_5(t-t_0)}} \quad (10)$$

Where k_5 is a constant and t_0 is the time at which the processing capability is half of its maximum.

Latency from device i to edge node j is modeled using a logarithmic function to reflect diminishing returns with increased distance as calculated in Eq. (11).

$$L_E^j(t) = L_{\text{base}} + k_6 \times \log(d(i,j)) \quad (11)$$

Where L_{base} is a base latency, k_6 is a constant, and $d(i,j)$ is the distance between device i and edge node j .

The storage capacity of edge node j can be modeled as Eq. (12).

$$S_{\text{total}}^j = S_{\text{base}}^j + \int_0^t k_7 \times \sin(k_8 \times \tau) d\tau \quad (12)$$

The integral term models periodic fluctuations in storage, where k_7 and k_8 are constants. The available bandwidth for edge node j at time t is influenced by a sinusoidal function to model periodic network congestion (Eq. (13)).

$$BW_E^j(t) = BW_{\text{base}}^j + k_9 \times \sin(k_{10} \times t) \quad (13)$$

Where BW_{base}^j is the base bandwidth, and k_9 and k_{10} are constants. Packet loss rate which is modeled in Eq. (14), using an inverse function.

$$PLR^j(t) = \frac{k_{11}}{BW_E^j(t) + k_{12}} \quad (14)$$

Where k_{11} and k_{12} are constants ensuring the stability of the function.

3.3. Cloud (C)

Centralized data centers are equipped with massive computational and storage resources. Cloud processing capability at time t is often vast but can be influenced by logarithmic growth due to technology advancements and infrastructure investments as modeled in Eq. (15).

$$P_{C(t)} = P_{\text{base}}^C \times \log(k_{13} \times t + 1) \quad (15)$$

Where P_{base}^C is the initial processing capability and k_{13} is a growth constant.

According to Eq. (16), latency from device i to the cloud is modeled using an exponential function to reflect the rapidly increasing latency with increased network congestion or distance.

$$L_C^i(t) = L_{\text{base}}^C \times e^{k_{14} \times d(i,C)} \quad (16)$$

Where L_{base}^C is a base latency, k_{14} is a constant, and $d(i,C)$ is the distance between device i and the cloud.

Cloud storage capacity can be modeled as Eq. (17), using a sigmoid function to depict the rapid initial growth that saturates over time as the technology matures.

$$S_C(t) = S_{\max}^C \times \frac{1}{1 + e^{-k_{15}(t-t_1)}} \quad (17)$$

Where $S_{\max}^C$ is the maximum achievable storage and t_1 is the time at which half of the maximum storage is achieved.

The reliability of the cloud, $R_C(t)$, is influenced by a sinusoidal variation to model periodic maintenance or updates as defined in Eq. (18).

$$R_{C(t)} = R_{\text{base}}^C + k_{16} \times \sin(k_{17} \times t) \quad (18)$$

Where R_{base}^C is the base reliability, and k_{16} and k_{17} are constants.

3.4. Interactions

3.4.1. D2D communication

D2D communication refers to direct data exchange between nearby mobile devices without routing through centralized network infrastructures like base stations or servers, enhancing efficiency and reducing latency. Let $P_D^i(t)$ be the processing capability of device i at time t and $P_{\text{req}}^i(t)$ be the processing requirement of the task for device i at time t . The spare computational resources, $S_{\text{spare}}^i(t)$, for device i is modeled as Eq. (19).

$$S_{\text{spare}}^i(t) = P_D^i(t) - P_{\text{req}}^i(t) \quad (19)$$

The total spare computational resources in the vicinity of device (i) is Eq. (20).

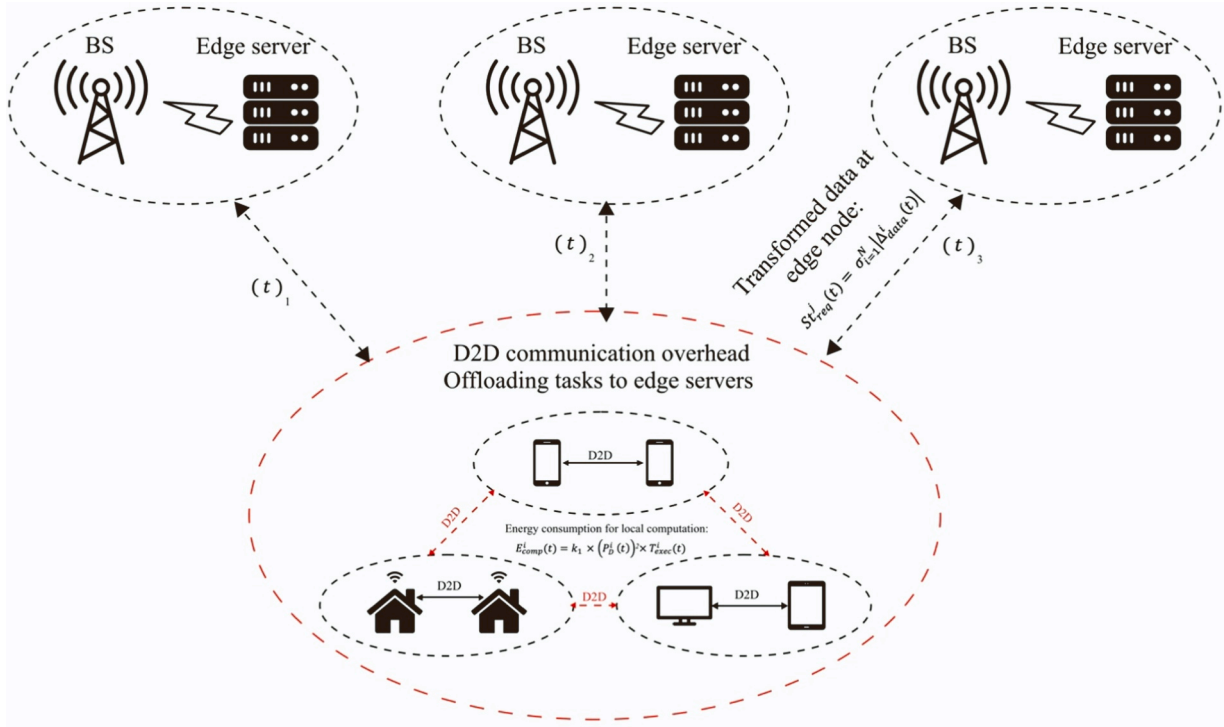


Fig. 4. Two-tier Sequential task offloading process to edge servers, depicting D2D communication and related energy consumption at each step.

$$S^i_{total_spare}(t) = \sum_{j \neq i} S^i_{total}(t) \times I(d(i,j) < d_{thresh}) \quad (20)$$

Where I is an indicator function, $d(i,j)$ represents the distance between devices i and j , and d_{thresh} is a threshold distance for D2D communication.

Let $L^{ij}_{D2D}(t)$ be the latency for D2D communication between devices i and j at time t . The average D2D latency for device i is calculated as Eq. (21).

$$L^i_{avg_D2D}(t) = \frac{1}{N-1} \sum_{j \neq i} L^{ij}_{D2D}(t) \quad (21)$$

Where N is the total number of devices in the vicinity of device i .

$BW^{ij}_{D2D}(t)$ be the bandwidth for D2D communication between devices i and j at time t and $E^{ij}_{trans}(t)$ be the energy consumed for the transmission. The efficiency of data transfer, $\eta^{ij}_{D2D}(t)$, according to Eq. (22), is then:

$$\eta^{ij}_{D2D}(t) = \frac{BW^{ij}_{D2D}(t)}{E^{ij}_{trans}(t)} \quad (22)$$

The overall efficiency for D2D communication for device i is Eq. (23).

$$\eta^i_{avg_D2D}(t) = \frac{1}{N-1} \sum_{j \neq i} \eta^{ij}_{D2D}(t) \quad (23)$$

Fig. 3, indicates the structure of single-tier D2D communication.

3.4.2. D2E communication

In the context of IoT architectures, D2E communication refers to the transfer of data and computational tasks from end-user devices directly to proximate edge computing nodes, which process information locally to reduce latency and network congestion. We define the computational intensity of a task at time (t) for the device (i) as presented in Eq. (24).

$$C^i_{intensity}(t) = \frac{P^i_{req}(t)}{P^i_D(t)} \quad (24)$$

Where $P^i_{req}(t)$ is the processing requirement of the task and $P^i_D(t)$ is the

device's processing capability. If $C^i_{intensity}(t)$ exceeds a threshold θ but is less than a value λ (which would justify cloud offloading), then D2E offloading is considered. Define the spare computational resources of edge node j as Eq. (25).

$$S^j_{spare}(t) = P^j_E(t) - P^j_{allocated}(t) \quad (25)$$

Where $P^j_E(t)$ is the processing capability of edge node j and $P^j_{allocated}(t)$ is the resources allocated to other tasks.

Similarly, the spare storage in edge node j is calculated in Eq. (26).

$$S^j_{spare}(t) = S^j_{total} - S^j_{used}(t) \quad (26)$$

Where S^j_{total} is the total storage of edge node j and $S^j_{used}(t)$ is the storage used at time t .

Define a preprocessing function φ that transforms the data from device i before offloading to the cloud. If the transformation is complex, it's more efficient to process at the edge, according to Eq. (27).

$$\Delta^i_{data}(t) = \varphi(data^i(t)) \quad (27)$$

The required storage for the transformed data at edge node j is defined as Eq. (28).

$$S^j_{req}(t) = \sigma^N_{i=1} |\Delta^i_{data}(t)| \quad (28)$$

Where σ is a summation over all devices N offloading to edge node j . Fig. 4, illustrates the structure of two-tier D2E communication.

3.4.3. D2C/E2C communication

D2C/E2C Communication refers to the process where devices or edge servers transmit data to the cloud for advanced processing, leveraging the cloud's vast computational resources to handle complex tasks beyond local capabilities. as Eq. (29) characterizes the computational intensity for a task at time t for entity I (either a device or an edge server).

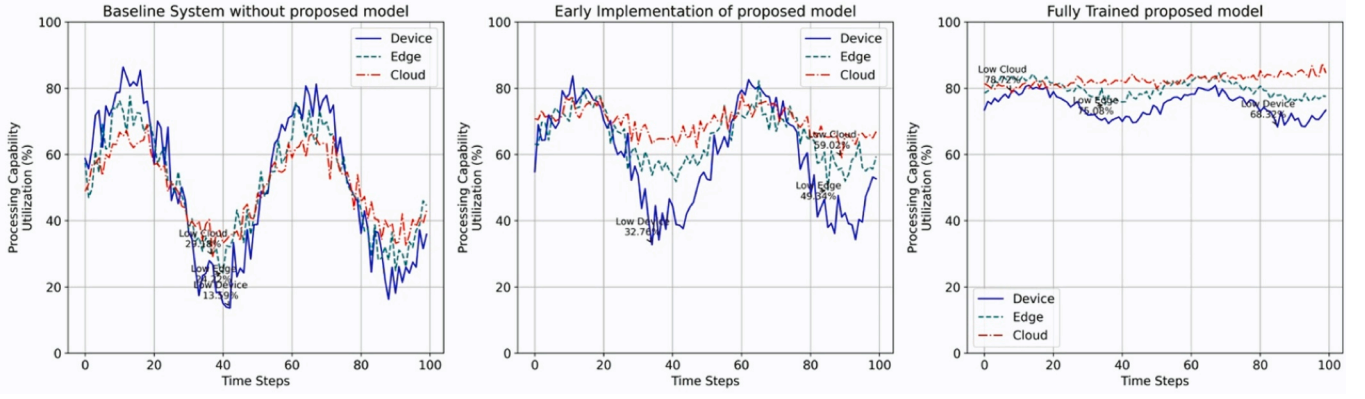


Fig. 5. Comparative analysis of processing capability utilization over time for baseline, early, and fully trained models.

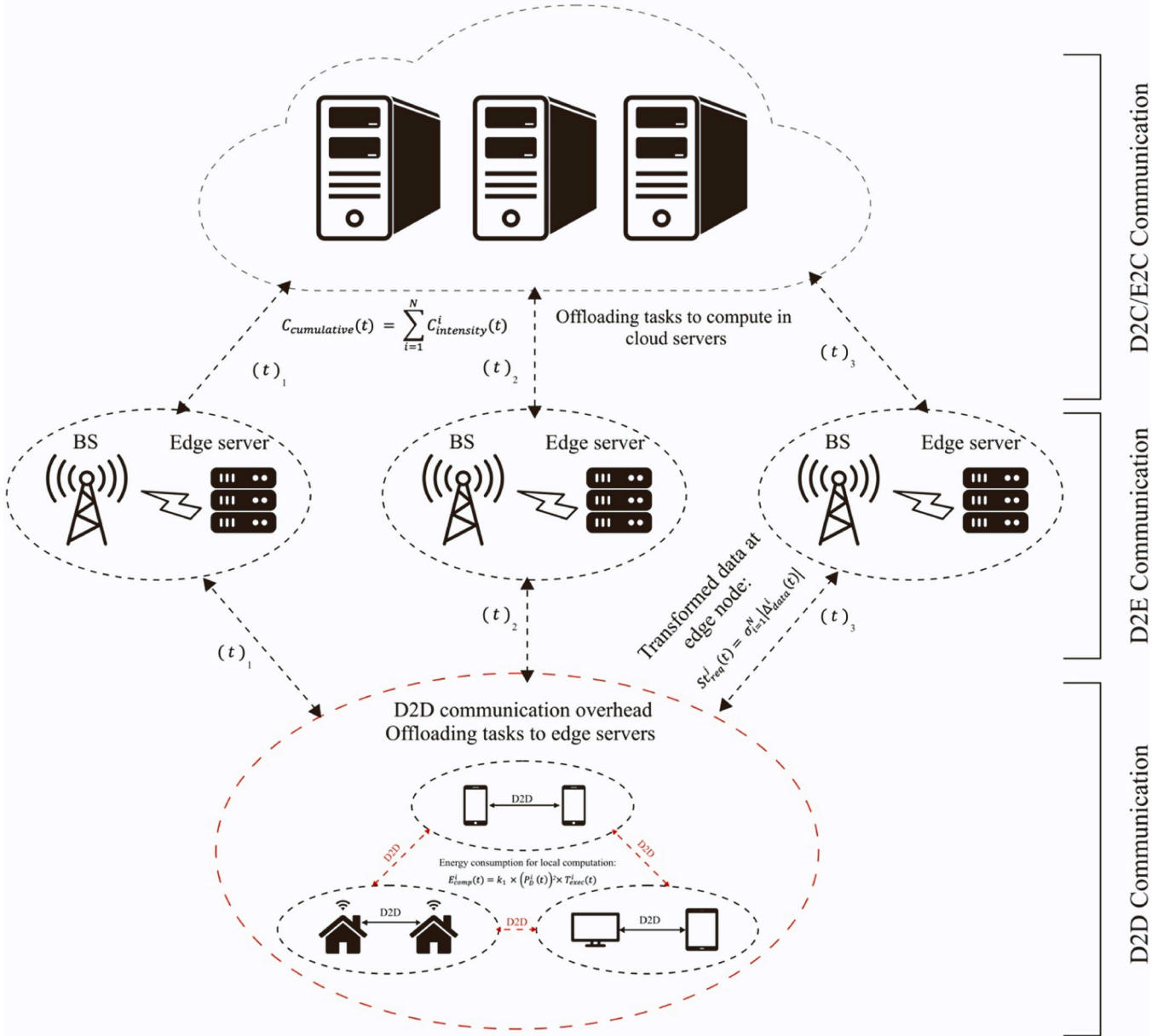


Fig. 6. Three-tier offloading process from devices to edge and cloud servers, highlighting D2D and D2E/D2C communication with related computations.

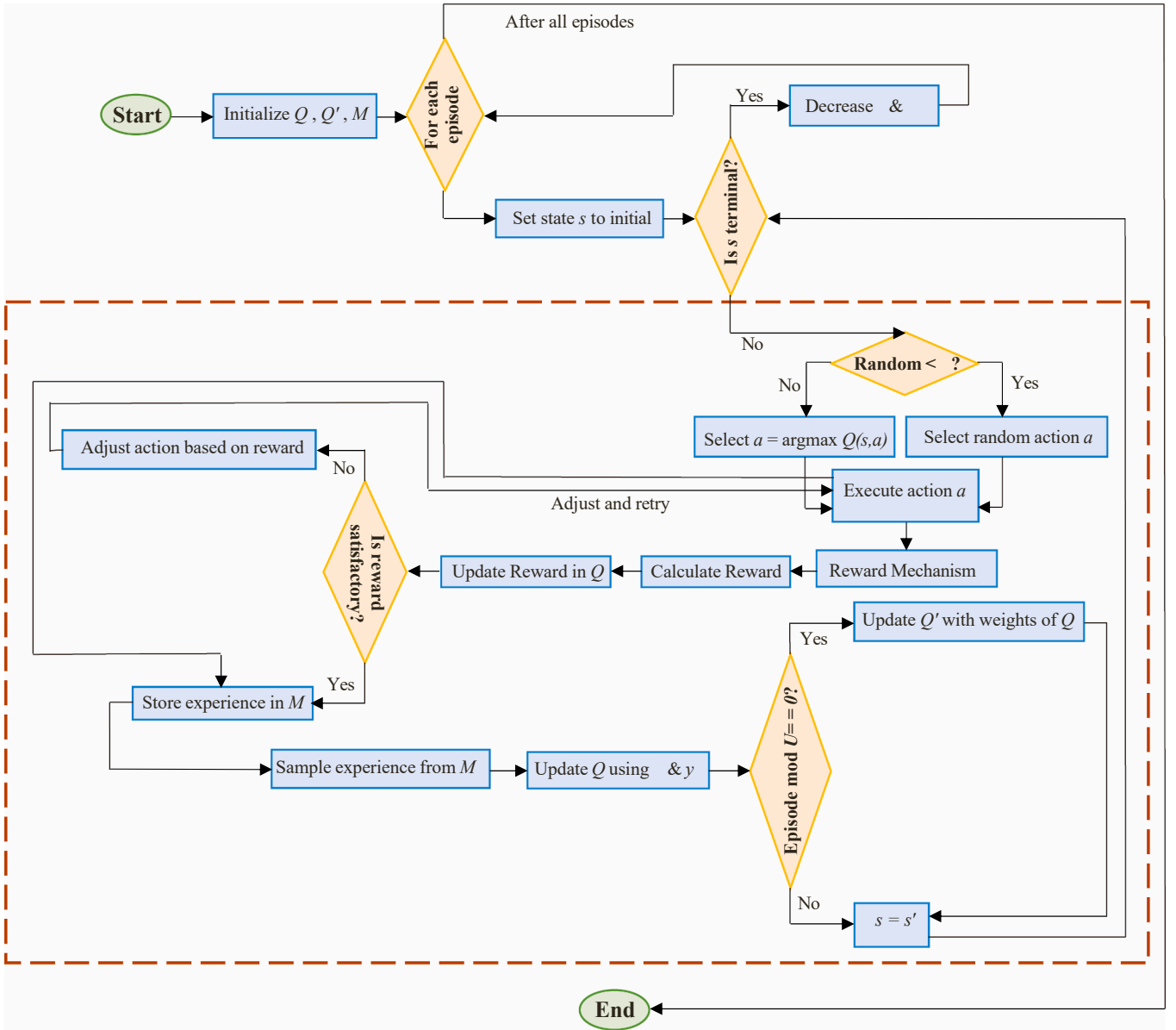


Fig. 7. Proposed algorithms flowchart depicting task offloading process, from input initialization to policy updating.

$$C_{intensity}^i(t) = \frac{P_{req}^i(t)}{P_i(t)} \quad (29)$$

Where $P_{req}^i(t)$ is the processing requirement of the task and $P_i(t)$ is the entity's processing capability. Fig. 5 displays how the processing power of devices, edge nodes, and cloud servers is optimized through the use of the proposed Rainbow DQN algorithm. It compares different stages of implementation, showing clear enhancements in how each tier manages computational load over time, demonstrating the algorithm's effectiveness in resource utilization across the system.

The cumulative computational intensity for all entities considering offloading to the cloud is calculated as Eq. (30).

$$C_{cumulative}(t) = \sum_{i=1}^N C_{intensity}^i(t) \quad (30)$$

Where N is the total number of entities (devices and edge nodes).

Let $L_{D2C/E2C}^i(t)$ be the latency from an entity I to the cloud at time t . If the integral of latency over a period T is within acceptable bounds, then offloading is considered as Eq. (31).

$$\int_0^T L_{D2C/E2C}^i(t) dt \leq L_{threshold} \quad (31)$$

Where $L_{threshold}$ is the maximum acceptable cumulative latency over the period T .

Let $R_C(t)$ denote the specialized resources (like GPU clusters) avail-

able in the cloud at time t . The decision to offload based on specialized resources can be modeled as Eq. (32).

$$\Omega^i(t) = \begin{cases} 1 & \text{if } R_c(t) > R_{\min} \text{ and } C_{\text{intensity}}^i(t) > \theta \\ 0 & \text{otherwise} \end{cases} \quad (32)$$

Where $\Omega(t)$ is the offloading decision for entity I at time t , $R_{\min}$ the minimum required specialized resources and θ is a threshold for computational intensity. Fig. 6, indicates the structure of three-tier D2C/E2C communication.

3.5. Rainbow DQN for task offloading

The challenge of task offloading in a D2D-Edge-Cloud computing system requires a robust decision-making mechanism to ensure optimal energy consumption, latency, and communication overhead. Rainbow DQN [30], an enhanced version of the standard DQN, offers a potent solution by amalgamating several advancements from the realm of DRL. At the heart of the Rainbow DQN is the Double DQN (DDQN) mechanism [31]. In traditional DQNs, there's a tendency to overestimate Q-values, which can lead to suboptimal offloading decisions. This overestimation might result in tasks being offloaded when it's not energy-efficient to do so or might increase the latency due to poor server selection. DDQN addresses this by decoupling the action selection from its evaluation. It achieves this by employing two networks: one for selecting the best action (the online network) and another for evaluating the chosen action (the target network). In the context of our system, this means that the IoT device, when deciding where to offload a task, gets a more accurate estimation of the outcomes of its actions, ensuring energy-efficient decisions that also optimize for latency. Next, the Prioritized Experience Replay mechanism ensures that the network learns more efficiently [32]. Traditional DQNs sample experiences (or transitions) uniformly from the memory. However, not all experiences contribute equally to the learning process. Some experiences, especially those where the network's predictions were off by a large margin, offer more learning value. By prioritizing the sampling of such experiences based on their Temporal Difference (TD) error, the network can focus on the most informative experiences. For task offloading, this means the system will quickly learn from any poor offloading decisions, adapting its strategy to prevent similar mistakes in the future. The Dueling DQN architecture further refines the decision-making process [33]. Instead of estimating the Q-value for each state-action pair directly, it splits the estimation into two streams: one for the state value function and another for the action advantage function. This separation allows the network to learn the value of being in a particular state without being clouded by the effects of each action. In the offloading scenario, this could help the IoT device understand the inherent benefits or challenges of its current situation (e.g., battery level, network congestion) before deciding on the best offloading strategy. In the task offloading scenario using Rainbow DQN, here's a step-by-step breakdown of the process:

Observation: The IoT device observes its current state, which includes its status, the nature of tasks awaiting offloading, and the conditions of the edge servers and cloud.

Decision: Based on its learned policy and current state, the IoT device uses the Rainbow DQN to decide where to offload the task – this could be another device, an edge server, or the cloud.

Action and Feedback: Upon deciding, the IoT device offloads the task. It then receives feedback in terms of energy consumed, latency experienced, and any communication overhead.

Learning: This feedback is used to compute the reward or penalty. The experience (state, action, reward, next state) is stored in the memory. The Rainbow DQN then samples from this memory, prioritizing experiences with high TD errors, and updates its weights to improve its policy.

Iteration: The process is iteratively refined as more tasks are offloaded, and the IoT device continues to learn and adapt its offloading strategy.

Algorithm 1 and Fig. 7, demonstrate a comprehensive explanation of how the proposed algorithm works. The flowchart illustrates the Rainbow DQN Task Offloading algorithm. Starting with initializations, it navigates through episodes, deciding actions based on exploration or Q-values. Upon executing an action, a reward mechanism evaluates the outcome, adjusting actions if unsatisfactory. The Q-values are updated periodically, refining decisions over iterations. Table 2, provides applied abbreviations in this section.

3.6. Problem formulation

Let's consider a set of IoT devices $D = d_1, d_2, \dots, d_M$ that can offload tasks to a set of edge servers $E = e_1, e_2, \dots, e_M$ or to the cloud C .

State Space: The states of an IoT device at time t is represented as Eq. (33).

$$s_t^i = (B_t^i, P_t^i, L_t^i, W_t^i) \quad (33)$$

where B_t^i is the battery level, P_t^i is the processing load, L_t^i is the latency, and W_t^i is the available bandwidth for device i .

Action Space. The action a denotes the offloading decision and is defined as Eq. (34).

$$a_t^i = \begin{cases} e_j & \text{C cloud} \\ 0 & \end{cases} \quad (34)$$

The objective is to minimize the energy consumption, latency, and communication overhead while ensuring the computational requirements are met. The cumulative reward R for device i is given by Eq. (35).

$$R_t^i = \sum_{\tau=0}^T (\omega_1 E_\tau^i + \omega_2 L_\tau^i - \omega_3 C_\tau^i) \quad (35)$$

where ω_1, ω_2 , and ω_3 are the weighting factors for energy, latency, and computation respectively; T is the time horizon.

The energy consumed by device i when offloading a task is calculated as Eq. (36).

$$E_t^i = \gamma P_t^i \int_0^t e^{-\beta \tau} d\tau \quad (36)$$

where γ is a device-specific constant and β is the decay rate.

The latency experienced by device i for task offloading is modeled as Eq. (37).

$$L_t^i = \alpha \log \left(\sum_{j=1}^M \frac{1}{d(j,i) + \delta} \right) \quad (37)$$

where $d(i,j)$ is the distance between device i and edge server j , and δ is a small constant to avoid division by zero.

The computational load on device (i) is given by Eq. (38).

$$P_t^i = \sum_{k=1}^K \lambda_k t^{\theta_k} \quad (38)$$

where K is the number of tasks, λ_k is the arrival rate of task k , and θ_k is the processing requirement of task k .

Algorithm 1. Rainbow DQN Task Offloading.

Algorithm 1. Rainbow DQN Task Offloading.

Input	Initial state (s) Environment model (D2D-Edge-Cloud system) Number of episodes (N) Learning rate (α) Discount factor (γ) Exploration rate (ϵ) Experience replay memory (M) Prioritization factor (β) Update frequency (U) for the target network
Output	Optimal policy (π) for task offloading

Begin

```

1 Initialize online Q-network (  $Q$  ) with random weights
2 Initialize target Q-network (  $Q'$  ) with weights of (  $Q$  )
3 Initialize experience replay memory (  $M$  ) to capacity (  $C$  )
4 for episode = 1 to (  $N$  ) do
5   state (  $s$  ) = initial state
6   while (  $s$  ) is not terminal do
7     if random () < (  $\epsilon$  ) then
8       Select random action (  $a$  )
9     else
10      Select (  $a = \arg \max_a Q (s, a)$  )
11    end if
12    Execute action (  $a$  ) and observe reward (  $r$  ) and next state (  $s'$  )
13    Store experience tuple (  $s, a, r, s'$  ) in (  $M$  ) with priority (  $p$  ) based on TD error
14    Sample a mini-batch of experiences from (  $M$  ) based on priority
15    for each experience (  $i, j, k, l$  ) in mini-batch do
16      if is terminal, then
17        (  $y = k$  )
18      else
19        (  $y = k + \gamma Q' (l, \arg \max_a Q (i, a))$  ) //DoubleQ – learning
20      end if
21      Update (  $Q$  ) using (  $\alpha$  ) and (  $y$  )
22    end for
23    if episode mod (  $U$  ) == 0 then
24      Update (  $Q'$  ) with weights of (  $Q$  )
25    end if
26    (  $s = s'$  )
27  end while
28  Decrease (  $\epsilon$  ) and (  $\beta$  ) (exploration and prioritization rates)
29 end for
```

According to Eq. (39), the available bandwidth for device i is.

$$W_t^i = W_{\max}^i \sin\left(\frac{\pi t}{T}\right) \quad (39)$$

The objective function to be maximized is the cumulative reward over the time horizon T is expressed as Eq. (40) to Eq (46).

$$\max_{a_t^i} \sum_{t=0}^T R_t^i \quad (40)$$

Subject to:

$$0 \leq B_t^i \leq B_{\max}^i \quad (41)$$

$$0 \leq P_t^i \leq P_{\max}^i \quad (42)$$

$$L_t^i \geq L_{\min}^i \quad (43)$$

$$W_t^i \geq 0 \quad (44)$$

$$\sum_{i=1}^N P_t^i \leq \sum_{j=1}^M P_{\max}^j + P_{\max}^C \quad (45)$$

$$a_t^i \in \{0\} \cup \mathcal{E} \cup \mathcal{C} \quad (46)$$

3.7. Complexity analysis

In this section, we analyze the computational complexity of the proposed Rainbow DQN task offloading algorithm. The complexity primarily arises from three components: the state space representation, the action space determination, and the neural network training.

State Space Complexity: Given that the states for an IoT device encompass its battery level, processing load, latency, and available bandwidth, the state space complexity is $O(N)$, where N is the number of IoT devices.

Action Space Complexity: The action space complexity is determined by the number of potential offloading targets (edge servers and the cloud). Given M edge servers and 1 cloud, the action space complexity

Table 2
Applied abbreviations.

Abbreviation	Explanation	Abbreviation	Explanation
D	Devices	Q_{online}	Online Q-Network used for action selection
P_D	Processing Capability of Devices	Q_{target}	Target Q-Network used for estimating future Q-values
E_{comp}	Energy usage of local computation	θ	Weights of the online Q-network
k_1	Device-specific constant	θ^-	Weights of the target Q-network
T_{exec}	Execution time for a task	D	Replay memory storing past experiences
R_{drain}	Rate of battery drain	N	Capacity of replay memory
k_2, k_3	Constants	P	Priority queue for sampling experiences based on TD-error
B_D	Battery life of a device	α	Prioritization weights for sampling from replay memory
Pos	Position of a device	ϵ	Epsilon value used for ϵ -greedy action selection
V	Velocity of a device	γ	Discount factor for future rewards in Q-value calculation
S_{used}	Storage used by a device at a particular time	C	Frequency for updating the target Q-network with an online network
BW	Bandwidth	T	Maximum number of steps in an episode
E	Edge Nodes	s	The current state of the system
P_E	Processing Capability of Edge Nodes	a	The action selected based on the current state
L_E	Latency from device to edge node	r	The reward received after executing the action
S_{total}	Total storage capacity of edge node	s'	Next state of the system after action execution
BW_E	The available bandwidth for the edge node	a'	The best action for the next state s' based on an online Q-network
PLR	Packet loss rate	y	Target Q-value for learning
C	Cloud	δ	Temporal Difference error between target and predicted Q-value
P_C	Processing Capability of Cloud	D2D	Device-to-Device Communication
L_C	Latency from the device to the cloud	S_{spare}	Spare Computational Resources
S_C	Cloud storage capacity	L_{D2D}	Average D2D latency for device
R_C	Reliability of the cloud	η_{D2D}	Efficiency of data transfer for D2D
D2E	Device-to-Edge Communication	D2C/E2C	Device/Edge to Cloud Communication
$C_{intensity}$	Computational intensity of a task	R	Cumulative reward
S_{spare}	Spare computational resources of the edge node	E	Energy consumed by the device when offloading a task
S_{spare}	Spare storage in edge node	L	Latency experienced by device for task offloading
Δ_{data}	Data transformed using the preprocessing function for offloading	W	The available bandwidth for the device
Q	Q-value or quality value in Rainbow DQN		

Table 3
Simulation Parameters.

Parameter	Value
The number of IoT devices	100, 200, 300
The number of edge nodes	10
The number of cloud servers	3
Server CPU for edge nodes	Intel i9-7920X
CPU Frequency (GHz) for edge nodes	4.3
Memory (GB) for edge nodes	64
Storage (GB) for edge nodes	2048
Network Bandwidth (Gbps) for edge nodes	100
The communication radius of each edge node	300 m
The D2D communication radius	30 m
The data bits of each task	[0.2, 4] MB
The computation demands of each task	[0.2, 4] GHz
The computing capabilities of each edge node	[6,15] GHz
The available computing resources of each edge node	[0.2, 0.5] F'm
The computing capabilities of each IoT device	[1,3] GHz
The transmission rates of IoT-to-edge network	[2,3] Mb/s
The transmission rates of edge-to-cloud network	[0.8, 1.5] Mb/s
Operating System	Ubuntu 18.04
Software Platform	Python 3.8
Deep Learning Library	TensorFlow
Containerization Platform	Docker
Workflow Management	SLURM

becomes $O(M+I)$. In practice, not all edge servers might be viable offloading targets due to factors like proximity or load. Thus, the average complexity becomes $O(M)$.

Neural Network Training Complexity: Training the Rainbow DQN requires backpropagation on a deep neural network. If L is the number of layers in the network, D is the average number of neurons in each layer, and E is the number of epochs for training, the complexity for training becomes $O(L \times D^2 \times E)$.

Overall Complexity: Combining the complexities from the state space, action space, and neural network training, the overall complexity of the Rainbow DQN task offloading algorithm is defined as Eq. (47).

$$O(N + M + L \times D^2 \times E) \quad (47)$$

It should be emphasized that the intricacies of state and action spaces pale in comparison to the demands of neural network training. Consequently, the scalability and computational expense of our proposed methodology primarily hinge on the neural network's architecture—the number of layers and neurons—and the extent of training, measured by epochs. Practically, this implicates that although the proposed Rainbow DQN algorithm is capable of accommodating a substantial array of IoT devices and edge servers, it is imperative to meticulously tailor the neural network design and training protocol to the deployment context to preclude excessive training durations.

4. Simulation environment and parameters

Our simulation environment was designed to capture the complexities of a D2D-Edge-Cloud system, focusing on the intricate interactions between IoT devices, edge nodes, and cloud servers. This setup allows for a comprehensive evaluation of the task offloading mechanisms proposed in our research. The simulated network includes a variable number of IoT devices (100, 200, or 300), with each device having computational capabilities ranging from 1 to 3 GHz. These devices generate tasks with data bits ranging from 0.2 to 4 MB and computation demands from 0.2 to 4 GHz. The tasks can be offloaded to the available edge nodes or directly to the cloud servers. 10 edge nodes are strategically placed within the network, each with a communication radius of 300 m and computing capabilities varying between 6 and 15 GHz. These edge nodes are responsible for handling offloaded tasks, with available computing resources ranging from 0.2 to 0.5 F'm. The edge nodes can further offload tasks to any of the three cloud servers if necessary. Communication between IoT devices can occur within a radius of 30 m, allowing for D2D interactions. The transmission rates for IoT-to-edge and edge-to-cloud communications are set between 2 and 3 Mb/s and 0.8–1.5 Mb/s, respectively. Detailed information and parameters are mentioned in Table 3.

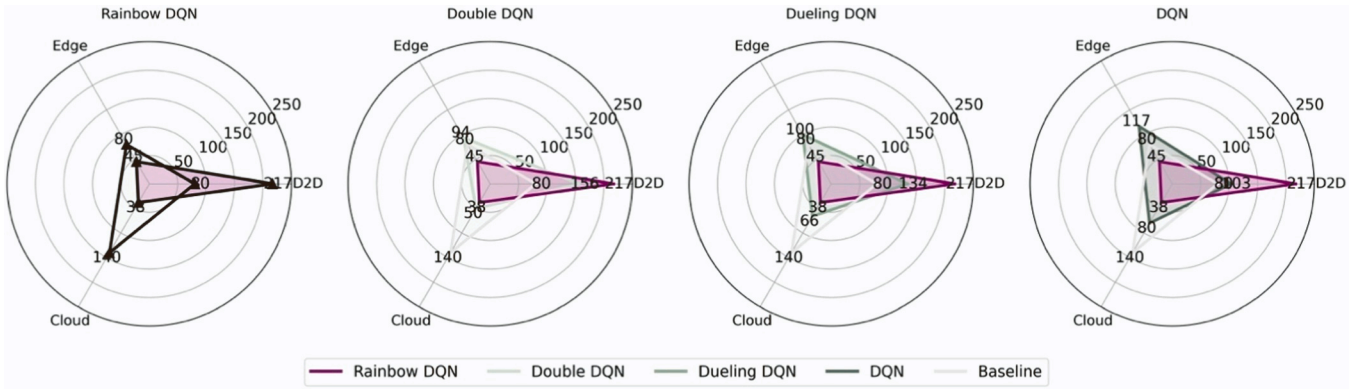


Fig. 8. Radar plot showcasing task distribution across D2D, edge, and cloud platforms for different algorithms.

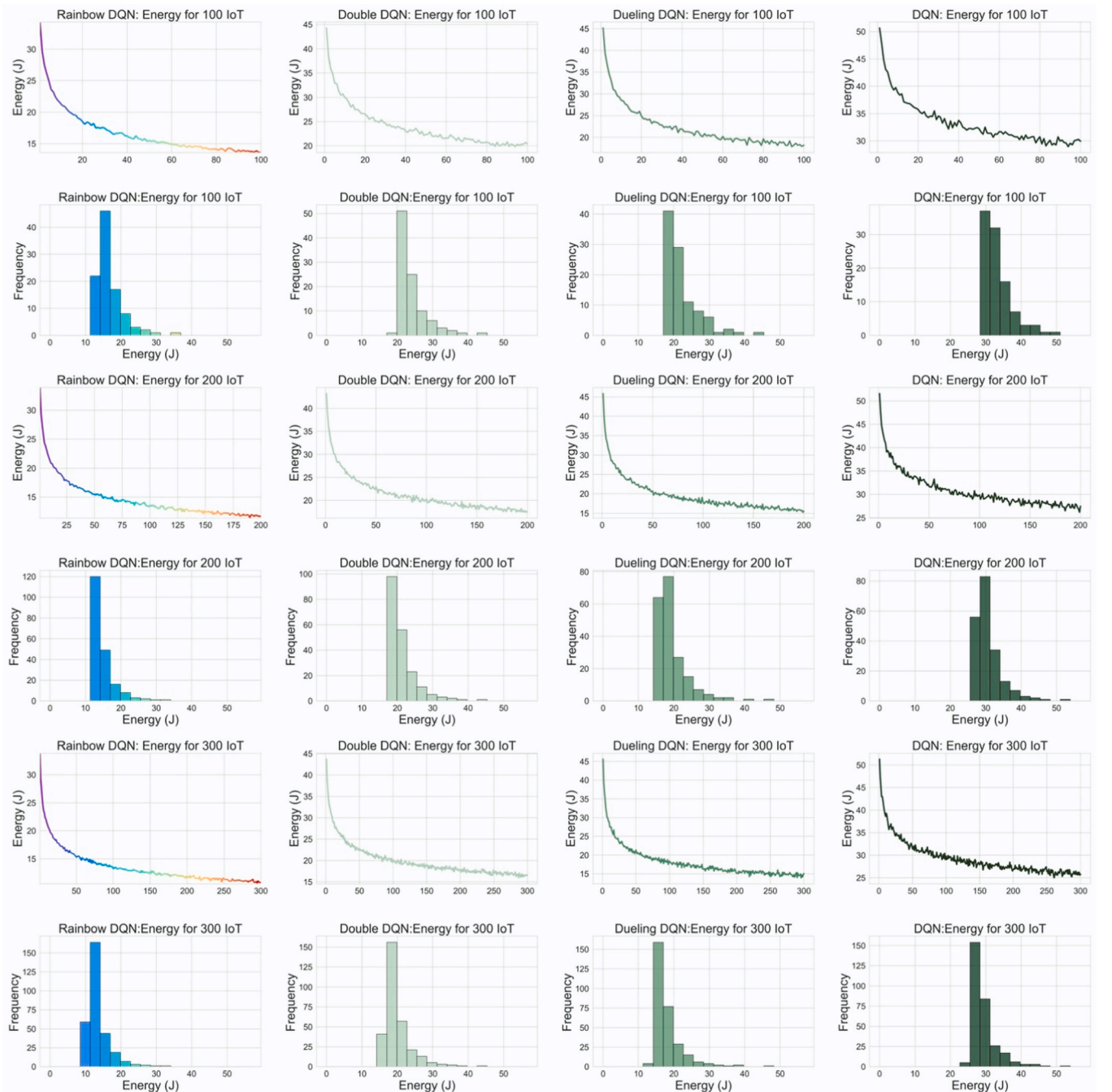


Fig. 9. The result of energy consumption of the proposed algorithm and comparison models in different scenarios.

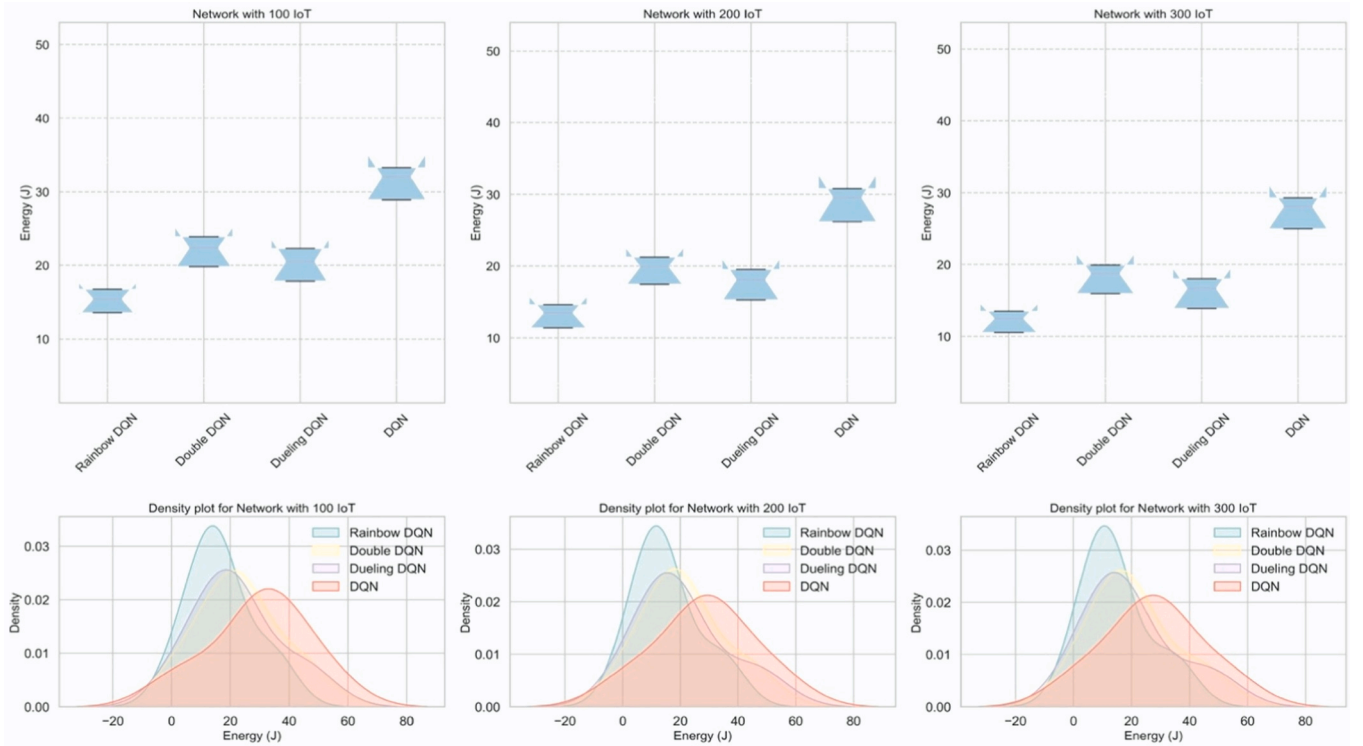


Fig. 10. Box plots and density of the energy consumption in different scenarios.

Table 4

The statistical analysis of energy consumption (J) for the algorithms in a network with 100 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	13.58445377	19.82386466	17.85601079	28.94800013
Max	34.2998889	44.28377672	45.1597109	50.65980184
Mean	16.74784466	23.88921228	22.27798531	33.27518459
50%	15.41480936	22.35288019	20.52275338	32.06118622
std.	3.671235946	4.345955888	4.839256308	4.079744974

Table 5

The statistical analysis of energy consumption (J) for the algorithms in a network with 200 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	11.38712271	17.45087345	15.27938554	26.19452558
Max	33.89822883	43.2193669	45.83624538	51.57242171
Mean	14.61569801	21.24415179	19.52345322	30.82242196
50%	13.50216962	19.91579342	18.07938688	29.55940525
Std.	3.343377979	3.992946366	4.466915405	3.914349237

5. Results and discussion

In this section, we present and analyze the results obtained from the simulation of our proposed Rainbow DQN model for task offloading in a D2D-edge-cloud computing environment. The results are compared against baseline and other DRL models, including Double DQN, Dueling DQN, and standard DQN.

Fig. 8 illustrates the task distribution across D2D, edge, and cloud platforms for various algorithms. The baseline shows a preference for cloud computing, offloading 140 tasks there while maintaining an equal distribution of 80 tasks each for D2D and edge. Rainbow DQN stands out by computing a substantial 217 tasks in D2D, minimizing latency and energy use, while only offloading 38 tasks to the cloud. Double DQN still favors D2D with 156 tasks but commits more to the cloud with 50 tasks

Table 6

The statistical analysis of energy consumption (J) for the algorithms in a network with 300 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	10.53533761	15.94147298	13.9048456	24.97733628
Max	33.84761715	43.70410433	45.61083149	51.29979551
Mean	13.51434314	19.89890182	18.00894206	29.29669699
50%	12.49999484	18.7219344	16.6567298	28.08919612
Std.	3.151973289	3.832734689	4.194189193	3.820733867

than Rainbow DQN. Dueling DQN presents a balanced approach, with 66 tasks in the cloud. Traditional DQN, while showing a preference for edge computing, also offloads a considerable 80 tasks to the cloud. The data emphasizes Rainbow DQN's efficiency in harnessing local resources, making it a promising candidate for real-world D2D-Edge-Cloud environments. The results also underscore the importance of selecting the right algorithm based on the desired balance between local and cloud computations.

As depicted in Fig. 9, we tested the energy consumption patterns across networks consisting of 100, 200, and 300 IoT devices for various algorithms. In all scenarios, Rainbow DQN distinctly outperformed the other algorithms in terms of energy conservation. Specifically, when comparing the average energy consumption of the Rainbow DQN to the next best performer, Double DQN, we noticed a significant percentage improvement in energy efficiency. This enhancement was approximately 29.8% for the 100-IoT network, 31.3% for the 200-IoT network, and 32% for the 300-IoT network. Additionally, when we analyzed the energy consumption against the DQN algorithm, the established baseline, the trend persisted. The percentage improvement of the Rainbow DQN over DQN was substantial. It was approximately 49.7% for the 100-IoT network, 52.5% for the 200-IoT network, and 53.8% for the 300-IoT network.

Fig. 10, showcases the box plots and density distributions that display the energy consumption patterns for different algorithms across networks with 100, 200, and 300 IoT devices. In the 100-IoT scenario,

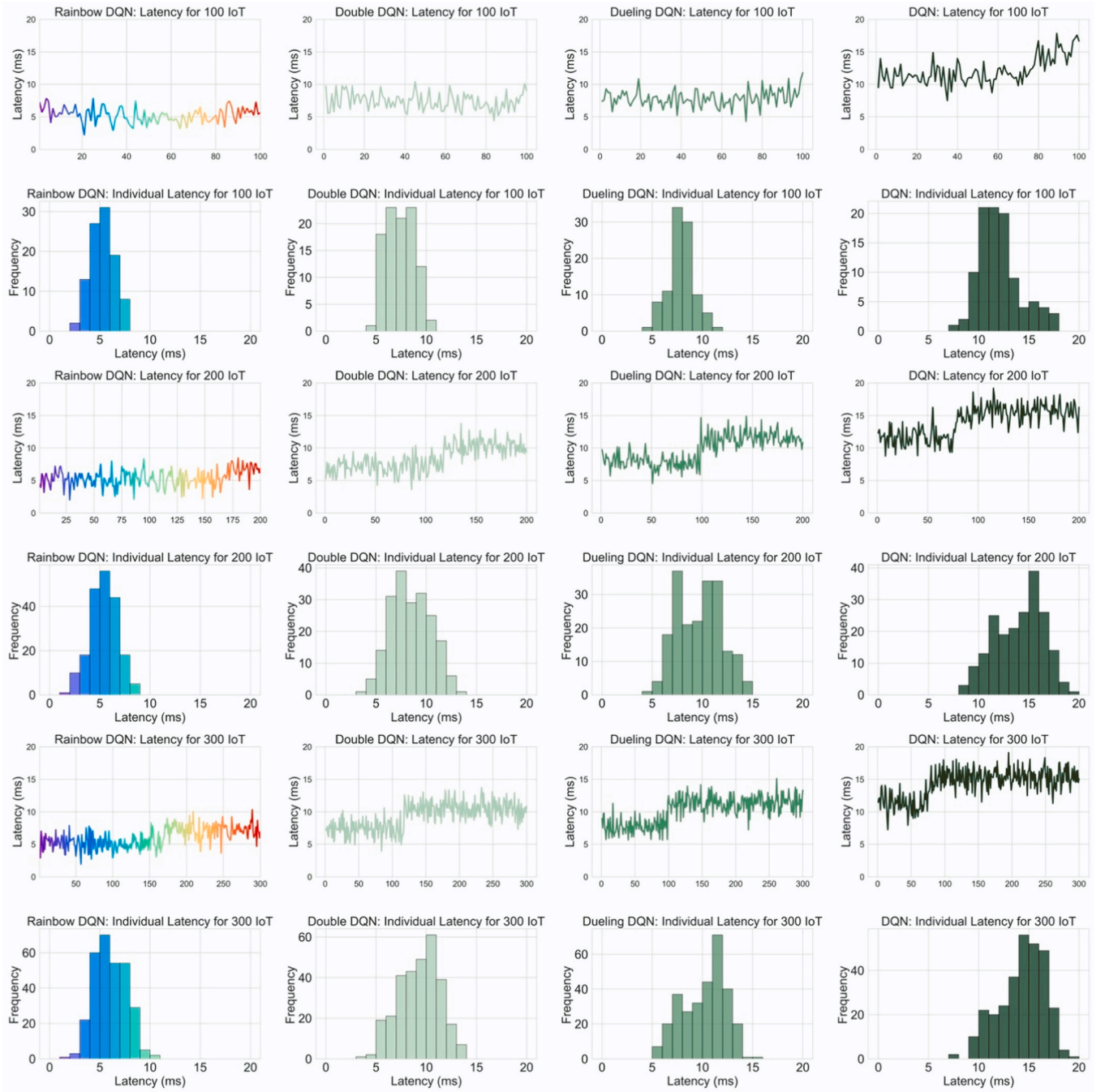


Fig. 11. The result of latency for the proposed algorithm and comparison models in different scenarios.

the Rainbow DQN algorithm emerges as a superior performer. It achieves a mean energy consumption of approximately 16.7478 J, which is substantially lower than DQN's mean of 33.2752 J. Notably, the maximum energy consumption with Rainbow DQN was capped at 34.2999 J, compared to DQN's maximum of 50.6598 J, indicating Rainbow DQN's efficiency in managing energy consumption. These insights are tabulated in Table 4. For the 200-IoT network, Rainbow DQN continues its impressive performance. It exhibits a mean energy consumption of 14.6157 J, which is notably lower than DQN's 30.8224 J. Furthermore, its maximum energy consumption remains controlled at 33.8982 J, which is less than DQN's peak energy consumption of 51.5724 J. This highlights Rainbow DQN's efficacy in larger network setups. Detailed figures can be found in Table 5. In the network setup of 300 IoT devices, Rainbow DQN maintains its trend of optimal

performance. It registers a mean energy consumption of 13.5143 J, which is less than DQN's mean of 29.2967 J. Also, its peak energy consumption is set at 33.8476 J, which is lower than the DQN's maximum of 51.2998 J. This underscores Rainbow DQN's consistent energy efficiency across dense networks. Comprehensive statistics are presented in Table 6. The accompanying density distributions further validate these observations, emphasizing the improved energy management of Rainbow DQN over its counterparts across all the tested scenarios.

Analyzing the latency results as shown in Fig. 11, across networks with varying IoT device quantities, Rainbow DQN consistently demonstrated superior performance in comparison to other algorithms. Focusing on the average latency, Rainbow DQN exhibited a significant reduction. In the 100-IoT network, the latency improvement was

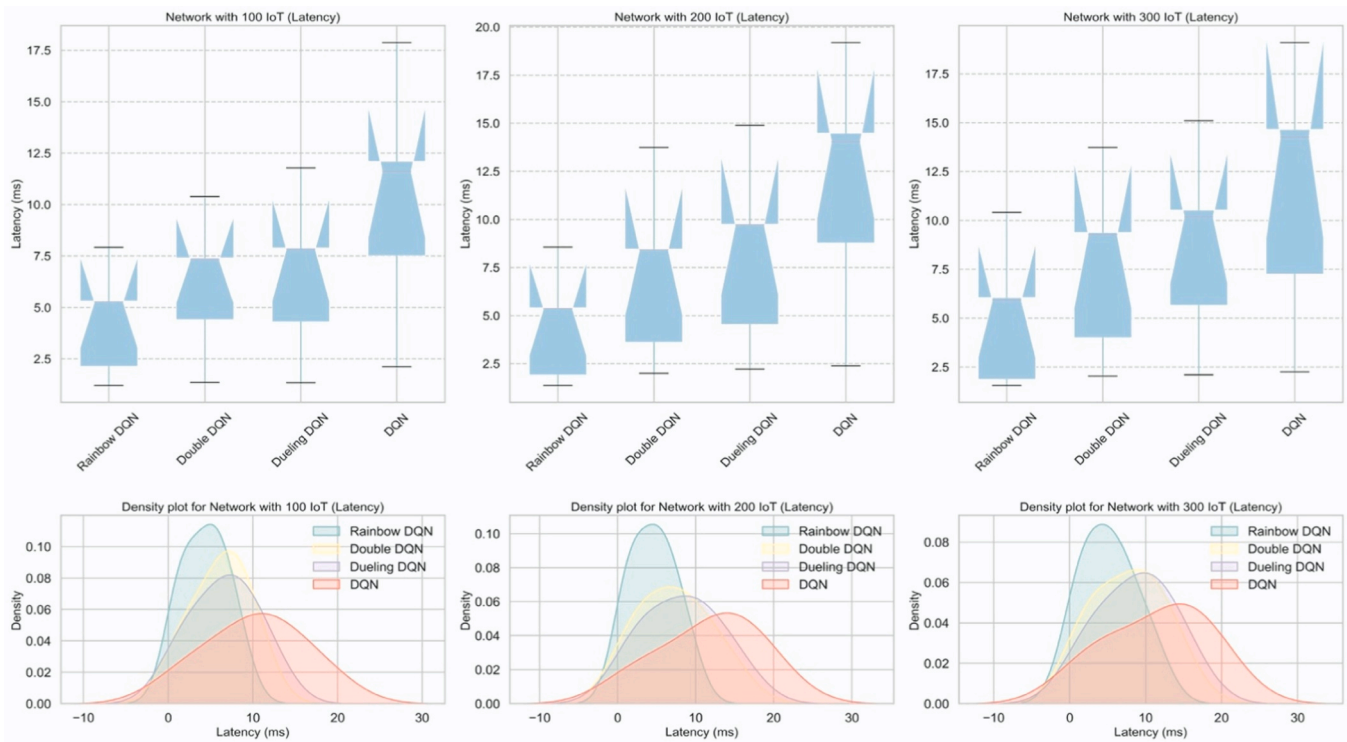


Fig. 12. Box plots and density of the latency in different scenarios.

Table 7

The statistical analysis of latency (ms) for the algorithms in a network with 100 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	2.136412221	4.409575902	4.295629417	7.511241643
Max	7.923705549	10.39808821	11.78107265	17.8722024
Mean	5.271769619	7.406616862	7.890998061	12.10180494
50%	5.312915343	7.332051316	7.838364109	11.60788493
Std.	1.215551723	1.351843296	1.335646515	2.120547443

Table 8

The statistical analysis of latency (ms) for the algorithms in a network with 200 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	1.912387399	3.615079143	4.542364377	8.772883554
Max	8.559695855	13.74636051	14.88716165	19.18955236
Mean	5.370702883	8.459943206	9.72219526	14.04261661
50%	5.403848515	8.422621386	9.747215558	14.47908333
Std.	1.37577821	1.990537746	2.218570072	2.390630352

approximately 27.5% compared to Double DQN and 56.5% compared to the baseline DQN. For the 200-IoT network, these improvements were 36.3% and 61.8%, respectively. Lastly, in the 300-IoT network, Rainbow DQN achieved a 34.7% reduction compared to Double DQN and a noteworthy 57.6% reduction compared to DQN. This consistent trend across various network sizes underlines the efficiency of Rainbow DQN in minimizing latency, thereby contributing to a faster and more responsive IoT network. The results indicate that the advanced techniques incorporated in Rainbow DQN, such as prioritized experience replay and double Q-learning, play a pivotal role in enhancing the network's performance, particularly in terms of reducing the time delay in data processing and task execution.

Fig. 12, provides a visual representation through box plots and density distributions, demonstrating the latency patterns of various

algorithms across networks consisting of 100, 200, and 300 IoT devices. In a network of 100 IoT devices, the Rainbow DQN algorithm achieves the most efficient performance. It registers a mean latency of approximately 5.2718 ms, which is significantly lower than DQN's mean latency of 12.1018 ms. Additionally, the highest latency with Rainbow DQN is capped at 7.9237 ms, while DQN reaches a maximum of 17.8722 ms. These results are elaborated in Table 7. Moving to the 200 IoT device scenario, Rainbow DQN continues to excel. It exhibits a mean latency of 5.3707 ms, which is markedly less than DQN's mean of 14.0426 ms. Furthermore, the maximum latency recorded for Rainbow DQN is 8.5597 ms, considerably less than DQN's peak latency of 19.1896 ms. Detailed statistics are available in Table 8. For the 300 IoT device setup, Rainbow DQN remains consistent in its performance. It showcases a mean latency of 6.0482 ms, which stands notably lower than DQN's average of 14.2720 ms.

In the examination of utility across different networks, as illustrated in Fig. 13, Rainbow DQN stands out, showcasing a substantial enhancement over the other algorithms. Within the 100-IoT network, Rainbow DQN surpassed Double DQN and DQN by 43.1% and 112.6% in terms of average utility, respectively. Moving to the 200-IoT network, the improvements were even more pronounced, with Rainbow DQN outperforming Double DQN and DQN by 49.4% and 125.2%, respectively. Lastly, in the 300-IoT network, Rainbow DQN maintained its superior performance, achieving a 50.0% improvement over Double DQN and a staggering 126.1% improvement over DQN.

These results underscore the effectiveness of the advanced techniques employed by Rainbow DQN in optimizing the overall utility of the network. The algorithm's ability to balance immediate rewards with long-term gains, while also efficiently utilizing network resources, translates into a more productive and cost-effective IoT ecosystem. The consistent trend across various network sizes attests to the robustness and scalability of Rainbow DQN, making it a preferable choice for complex IoT environments aiming to maximize utility. Moreover, its peak latency is recorded at 10.4167 ms, which is comparatively less than the DQN's maximum latency of 19.0965 ms. The detailed results are documented in Table 9. The density distributions accompanying the box

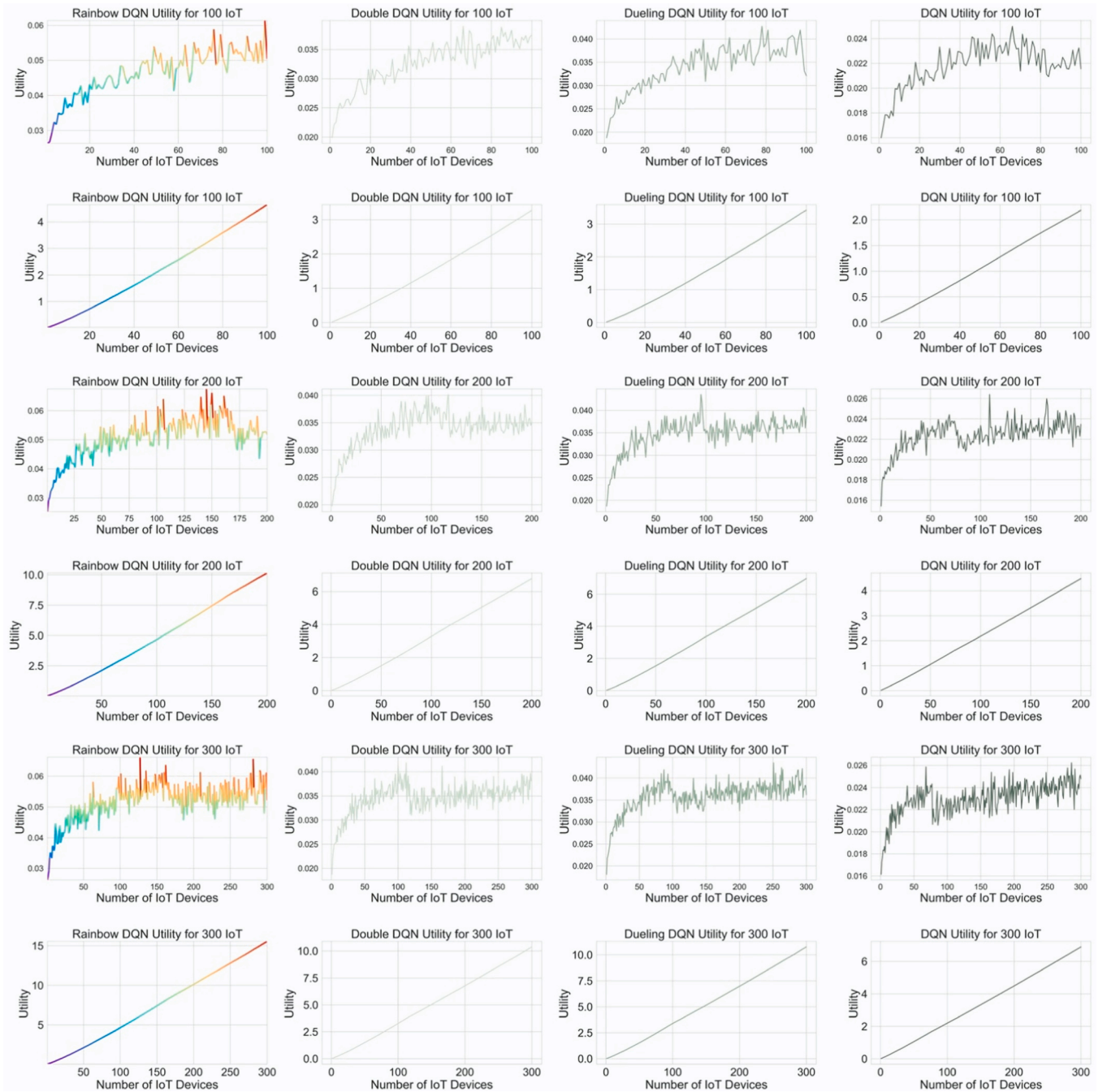


Fig. 13. The result of utility for the proposed algorithm and comparison models in different scenarios.

Table 9

The statistical analysis of latency (ms) for the algorithms in a network with 300 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	1.867169025	3.99638955	5.659803274	7.251982195
Max	10.41668501	13.7349741	15.10782511	19.0964829
Mean	6.048198497	9.27667239	10.18790116	14.27201469
50%	5.92104831	9.37773264	10.52797526	14.65298127
Std.	1.562859874	2.034654792	2.106777764	2.25302229

plots further emphasize Rainbow DQN's superior latency management over its counterparts in all examined scenarios.

Fig. 14, presents the utility statistics for various algorithms in

networks with 100, 200, and 300 IoT devices. The box plots and density distributions give a comprehensive view of the utility patterns associated with each algorithm across the different network sizes. In the 100 IoT scenarios, Rainbow DQN showcases impressive utility performance. With a mean utility value of 0.046862294, it outperforms the DQN algorithm, which has a mean of 0.022049451. The maximum utility achieved by Rainbow DQN, 0.06010481, is considerably higher than DQN's 0.025500162. This trend of Rainbow DQN's superiority is further affirmed by examining the standard deviations and median values detailed in Table 10. As the network size is increased to 200 IoT devices, Rainbow DQN continues its commendable performance. It records a mean utility of 0.050575777, which is notably higher than DQN's mean of 0.022477765. The highest utility score for Rainbow DQN stands at 0.068025523, far surpassing the DQN's peak of 0.026232611. This data, indicated in Table 11, underscores the efficiency and utility

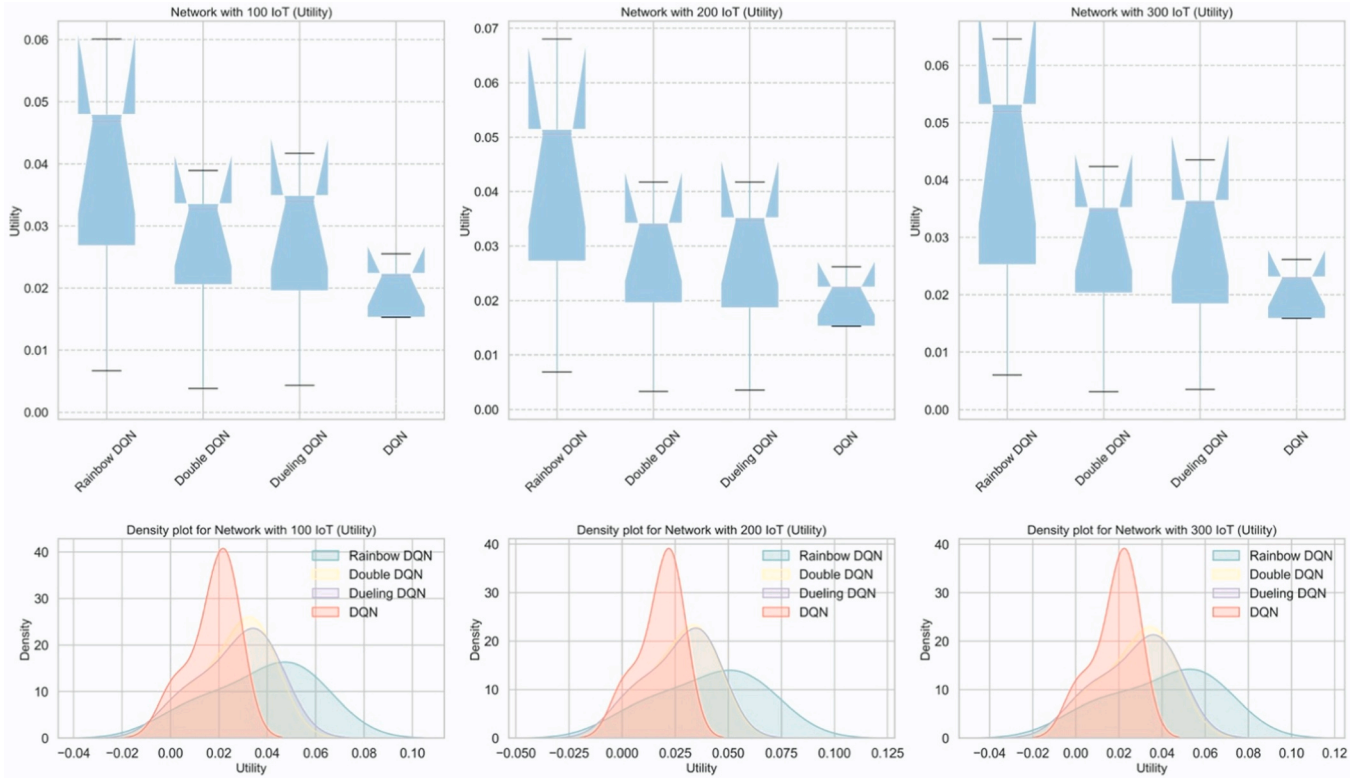


Fig. 14. Box plots and density of the utility in different scenarios.

Table 10

The statistical analysis of utility for the algorithms in a network with 100 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	0.026822416	0.02053462	0.019564489	0.015324217
Max	0.06010481	0.038969696	0.041672512	0.025500162
Mean	0.046862294	0.032739864	0.033950548	0.022049451
50%	0.047982744	0.033579408	0.034899386	0.022374887
Std.	0.006672915	0.003799366	0.004330619	0.001765066

Table 11

The statistical analysis of utility for the algorithms in a network with 200 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	0.027258094	0.019596427	0.018694296	0.015314341
Max	0.068025523	0.041744331	0.041743683	0.026232611
Mean	0.050575777	0.033856321	0.034724671	0.022477765
50%	0.051441686	0.034226394	0.035241275	0.022456857
Std.	0.006859817	0.003304516	0.003514234	0.001507442

Table 12

The statistical analysis of utility for the algorithms in a network with 300 IoT.

	Rainbow DQN	Double DQN	Dueling DQN	DQN
Min	0.025281098	0.020298306	0.018413562	0.015899081
Max	0.064553615	0.042372555	0.043544448	0.026184245
Mean	0.05196755	0.034676221	0.035966484	0.023003752
50%	0.053177244	0.035230378	0.036428878	0.02313415
Std.	0.006005475	0.003115219	0.003483543	0.001450138

management of the Rainbow DQN in medium-sized networks. In larger networks with 300 IoT devices, Rainbow DQN sustains its trend of high performance. It registers a mean utility value of 0.05196755, significantly outdoing DQN's mean of 0.023003752. Furthermore, its maximum utility value is 0.064553615, still considerably ahead of the

DQN's maximum of 0.026184245. Table 12, provides a comprehensive overview of the utility statistics, highlighting the consistency of Rainbow DQN's performance even in dense networks.

6. Conclusion and future works

In this paper, we have introduced a comprehensive task offloading framework within the D2D-Edge-Cloud environment, leveraging the capabilities of the Rainbow DQN, a sophisticated variant of DRL. Our approach stands out in its ability to make intelligent offloading decisions across the three tiers, optimizing the balance between D2D, D2E, and D2C/E2C communications. This not only enhances the performance of end devices but also optimizes the utilization of edge and cloud resources. Our extensive simulations and comparative analysis with state-of-the-art algorithms have demonstrated significant improvements in energy efficiency, latency, and utility, underscoring the efficacy of our proposed solution. However, there are some limitations to our current framework. The computational complexity introduced by the Rainbow DQN, which may lead to increased overhead, requires careful consideration. Additionally, the dependency on consistent and high-quality data for training the DRL model poses challenges in environments where data may be noisy or sparse. The real-world deployment of our framework could also face limitations in terms of infrastructure and technology variability across different regions. Moreover, the integration of security and privacy measures in the offloading process has not been deeply explored in our current framework. To overcome the limitations identified in our current framework, future research will be directed toward optimizing the computational efficiency of the Rainbow DQN algorithm. Furthermore, future work will focus on adapting the framework to the diverse infrastructural and technological contexts found in real-world deployments. Moreover, introducing robust security and privacy measures in the task offloading process, while maintaining efficiency, is a challenge we are eager to tackle. By addressing these areas, we aim to push our framework closer to real-world applicability and broader adoption in the IoT landscape.

Ethical Approval

The authors of this paper declare that they have completely avoided publishing ethics, including plagiarism, misconduct, falsification of data, or double submission and publication, about the publication of the presented article, and there is no commercial in this regard and the authors have not received any money for presenting their paper. The corresponding author also declares that this paper has not been published elsewhere and has not been submitted to another publication at the same time.

Competing interests

There are not any competing interests in this paper.

Funding

Authors have not received any money for presenting their paper.

CRediT authorship contribution statement

Komeil Moghaddasi: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Shakiba Rajabi:** Writing – original draft, Software, Investigation, Formal analysis, Data curation, Conceptualization. **Farhad Soleimanian Gharehchopogh:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization. **Ali Ghaffari:** Writing – original draft, Validation, Conceptualization, Supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

References

- [1] S. Abdullah, Saifullah, A.O. Almagrabi, An integrated group decision-making framework for the evaluation of artificial intelligence cloud platforms based on fractional fuzzy sets, *Mathematics* vol. 11 (21) (2023) 4428.
- [2] M. Adil, et al., Healthcare Internet of Things: security threats, challenges and future research directions, *IEEE Internet Things J.* (2024).
- [3] Y. Zhang, S. Abdullah, I. Ullah, F. Ghani, A new approach to neural network via double hierarchy linguistic information: Application in robot selection, *Eng. Appl. Artif. Intell.* vol. 129 (2024) 107581, <https://doi.org/10.1016/j.engappai.2023.107581>.
- [4] M.D. Zia, F. Yousafzai, S. Abdullah, K. Hila, Complex linear Diophantine fuzzy sets and their applications in multi-attribute decision making, *Eng. Appl. Artif. Intell.* vol. 132 (2024) 107953, <https://doi.org/10.1016/j.engappai.2024.107953>.
- [5] M. Shams, S. Abdullah, Selection of best industrial waste management technique under complex non-linear Diophantine fuzzy Dombi aggregation operators, *Appl. Soft Comput.* vol. 148 (2023) 110855, <https://doi.org/10.1016/j.asoc.2023.110855>.
- [6] F.S. Gharehchopogh, B. Abdollahzadeh, S. Barshandeh, B. Arasteh, A multi-objective mutation-based dynamic Harris Hawks optimization for botnet detection in IoT, *Internet Things* vol. 24 (2023) 100952.
- [7] N. Karankar, A. Seth, A Comprehensive Survey on Internet of Things Security: Challenges and Solutions, *Mob. Comput. Sustain. Inform.: Proc. ICMCSI 2023* (2023) 711–728.
- [8] J. Sharma, P.S. Mehra, "Secure communication in IOT-based UAV networks: A systematic survey," *Internet Things* (2023) 100883.
- [9] K. Lone, S.A. Sofi, A review on offloading in fog-based Internet of Things: Architecture, machine learning approaches, and open issues, *High. -Confid. Comput.* (2023) 100124.
- [10] K. Moghaddasi, S. Rajabi, F.S. Gharehchopogh, Multi-Objective Secure Task Offloading Strategy for Blockchain-Enabled IoV-MEC Systems: A Double Deep Q-Network Approach, *IEEE Access* (2024).
- [11] G. Baranwal, D. Kumar, D.P. Vidyarthi, Blockchain based resource allocation in cloud and distributed edge computing: a survey, *Comput. Commun.* (2023).
- [12] S. Lu, J. Lu, K. An, X. Wang, Q. He, Edge computing on IoT for machine signal processing and fault diagnosis: A review, *IEEE Internet Things J.* (2023).
- [13] K. Moghaddasi, M. Masdari, Blockchain-driven optimization of IoT in mobile edge computing environment with deep reinforcement learning and multi-criteria decision-making techniques, *Clust. Comput.* (2023), <https://doi.org/10.1007/s10586-023-04195-4>.
- [14] K. Moghaddasi, S. Rajabi, F.S. Gharehchopogh, An enhanced asynchronous advantage actor-critic-based algorithm for performance optimization in mobile edge computing -enabled internet of vehicles networks, *Peer-to-Peer Netw. Appl.* (2024), <https://doi.org/10.1007/s12083-024-01633-x>.
- [15] F.S. Abkenar, et al., A Survey on Mobility of Edge Computing Networks in IoT: State-of-the-Art, Architectures, and Challenges, *IEEE Commun. Surv. Tutor.* (2022).
- [16] K. Moghaddasi, S. Rajabi, F. Soleimanian Gharehchopogh, M. Hosseinzadeh, An Energy-Efficient Data Offloading Strategy for 5G-Enabled Vehicular Edge Computing Networks Using Double Deep Q-Network, *Wirel. Pers. Commun.* vol. 133 (3) (2023) 2019–2064, <https://doi.org/10.1007/s11277-024-10862-5>.
- [17] M. Jangjou, M.K. Sohrabi, A comprehensive survey on security challenges in different network layers in cloud computing, *Arch. Comput. Methods Eng.* vol. 29 (6) (2022) 3587–3608.
- [18] S. Ahmad, I. Shakeel, S. Mehrez, J. Ahmad, Deep learning models for cloud, edge, fog, and IoT computing paradigms: Survey, recent advances, and future directions, *Comput. Sci. Rev.* vol. 49 (2023) 100568.
- [19] Z. Su, et al., Energy-efficiency optimization for d2d communications underlying uav-assisted industrial iot networks with swipt, *IEEE Internet Things J.* vol. 10 (3) (2022) 1990–2002.
- [20] M. Ibrar, et al., Adaptive capacity task offloading in multi-hop D2D-based social industrial IoT, *IEEE Trans. Netw. Sci. Eng.* (2022).
- [21] X. Liu, et al., Joint Resource Allocation and Drones Relay Selection for Large-Scale D2D Communication Underlying Hybrid VLC/RF IoT Systems, *Drones* vol. 7 (9) (2023) 589.
- [22] Q. Guo, F. Tang, N. Kato, Federated Reinforcement Learning-Based Resource Allocation for D2D-Aided Digital Twin Edge Networks in 6G Industrial IoT, *IEEE Trans. Ind. Inform.* (2022).
- [23] Y. Yang, C. Long, J. Wu, S. Peng, B. Li, D2D-enabled mobile-edge computation offloading for multiuser IoT network, *IEEE Internet Things J.* vol. 8 (16) (2021) 12490–12504.
- [24] S.A. Gbadamosi, G.P. Hancke, A.M. Abu-Mahfouz, Interference-aware and coverage analysis scheme for 5G NB-IoT D2D relaying strategy for cell edge QoS improvement, *IEEE Internet Things J.* (2023).
- [25] Y. Jia, F. Zhong, A. Alrawais, B. Gong, X. Cheng, Flowguard: An intelligent edge defense mechanism against IoT DDoS attacks, *IEEE Internet Things J.* vol. 7 (10) (2020) 9552–9562.
- [26] I. Hafeez, M. Antikainen, A.Y. Ding, S. Tarkoma, IoT-KEEPER: Detecting malicious IoT network activity using online traffic analysis at the edge, *IEEE Trans. Netw. Serv. Manag.* vol. 17 (1) (2020) 45–59.
- [27] Y. Chen, Y. Sun, C. Wang, T. Taleb, Dynamic task allocation and service migration in edge-cloud iot system based on deep reinforcement learning, *IEEE Internet Things J.* vol. 9 (18) (2022) 16742–16757.
- [28] T. Cui, R. Yang, C. Fang, S. Yu, Deep Reinforcement Learning-Based Resource Allocation for Content Distribution in IoT-Edge-Cloud Computing Environments, *Symmetry* vol. 15 (1) (2023) 217.
- [29] T.-M. Pham, Optimization of resource management for nfv-enabled iot systems in edge cloud computing, *IEEE Access* vol. 8 (2020) 178217–178229.
- [30] M. Hessel, et al., Rainbow: Combining improvements in deep reinforcement learning, *Proc. AAAI Conf. Artif. Intell.* vol. 32 (1) (2018).
- [31] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, *Proc. AAAI Conf. Artif. Intell.* vol. 30 (1) (2016).
- [32] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2015.
- [33] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, N. Freitas, Dueling network architectures for deep reinforcement learning, in *International conference on machine learning*, PMLR, 2016, pp. 1995–2003.